Milestone 4

Scale the Marketplace

Build-a-Store Semester Project | Team of 2 | Two weeks

GOAL	FOCUS	GRADING
Scale product sorting, selection, and lookup.	Merge Sort, Quick Sort, QuickSelect, Mapping & Hashing	60 auto + 40 manual

1. Project Overview

Milestone 3 made the product catalog searchable, sortable, and more efficient. In Milestone 4, the marketplace grows again: you will move from quadratic sorting and binary search to divide-and-conquer algorithms and hash-based lookup.

You will implement Merge Sort, Quick Sort, and QuickSelect, then build a HashMapping that stores key-value pairs using buckets, handles collisions with separate chaining, and rehashes as the table grows. The Store will use these new tools to organize products, select products by rank, and find products by ID efficiently.

This milestone is cumulative. Keep all Milestone 3 behavior working while adding the new algorithms and mapping structure. The goal is to understand not only what result an algorithm produces, but why the algorithm and data structure are appropriate for the problem.

The progression
Quadratic sorting → divide and conquer → faster selection and lookup with hashing.

2. What Stays the Same and What Changes

From Milestone 3	Milestone 4
All M3 classes and behavior remain required.	Add Merge Sort, Quick Sort, QuickSelect, Entry, and HashMapping.
Store keeps its products, customers, orders, cart, queue, stack, and existing search/sort behavior.	Store gains divide-and-conquer sorting, QuickSelect, and hash-based product lookup.
algorithms.py remains the home of required algorithms.	Store methods must call the corresponding functions from algorithms.py and use HashMapping for product lookup.
Do not break previous milestone behavior.	Add the new functionality without replacing required M3 methods.

3. New Algorithms and Their Purpose

Algorithm / Structure	Purpose
Merge Sort	Sort products using divide, conquer, and combine. The required result is ascending product ID order.
Quick Sort	Sort products by choosing a pivot, partitioning the list, and recursively sorting both sides.
QuickSelect	Find the product at a requested rank without fully sorting the list. Recurse into only the side that can contain the requested item.
Entry	Store one key-value pair for the mapping.
HashMapping	Implement the Mapping ADT with buckets, a hash function, separate chaining, and rehashing.

4. Required Classes, Functions, and Methods

Follow the function and method names, parameters, and return types exactly. You may add helper functions, but do not rename or remove the required methods.

4.1 New algorithms.py

Put the three divide-and-conquer functions below in algorithms.py. Do not use Python's built-in sort() or sorted(). Merge Sort and Quick Sort must be implemented as divide-and-conquer algorithms. QuickSelect must recurse into only the partition that can contain the requested rank.

Function	Purpose	Return type
merge_sort(products)	Sort the Product list in place in ascending order by product ID using Merge Sort. Use a base case, divide the list, recursively sort the halves, and combine them with a merge step.	None
quick_sort(products)	Sort the Product list in place in ascending order by product ID using Quick Sort. Choose a pivot, partition the list, and recursively sort the left and right parts.	None
quick_select(products, k)	Return the kth smallest Product by product ID, where k = 1 represents the smallest product, k = 2 represents the second smallest, and so on. Use QuickSelect by partitioning the products and recursively searching only the side that contains the kth smallest product. Return None if k < 1 or k > len(products).	Product | None

Divide-and-conquer checklist: identify the base case, divide the problem, solve smaller subproblems recursively, and combine or finish using the partition result. The Module 7 materials describe these as the three main steps plus a base case.

4.2 Entry and HashMapping

HashMapping implements the Mapping ADT. Keys must be hashable; for this project, Product IDs are strings. Different keys can map to the same bucket, so collisions must be handled with separate chaining.

Class / Method	Purpose	Return type
Entry.__init__(key, value)	Create an Entry that stores one key-value pair.	None
HashMapping.__init__(size=2)	Create the bucket table and initialize the number of stored key-value pairs.	None
put(key, value)	Add a new key-value pair or update the value for an existing key. Use a hash to choose a bucket.	None
get(key)	Return the value associated with key. Raise KeyError if the key is not present.	object
__contains__(key)	Return True when key is stored in the mapping; otherwise False.	bool
__len__()	Return the number of key-value pairs currently stored.	int
rehash()	Create a larger bucket table and reinsert all existing entries using the new table size.	None

Collision rule: when two different keys map to the same bucket, keep both entries in that bucket rather than overwriting either one. Rehashing must recompute bucket positions because the table size changes.

4.3 Store Updates

All Milestone 3 methods remain required. In store.py, import the required functions from algorithms.py. Each new Store method must call the corresponding function from algorithms.py. The Store should also maintain a HashMapping for products so find_product() uses hash-based lookup instead of the old linear scan.

Method	Purpose	Return type
Store.__init__()	Keep all M3 collections and initialize an empty HashMapping for product lookup.	None
add_product(product)	Keep the existing M3 behavior. When the product is successfully added, also put the product into the HashMapping using its product ID as the key.	bool
find_product(product_id: str)	Use the Store HashMapping to find and return the Product for the given ID. Return None when the ID is not present. Do not scan self.products linearly.	Product | None
merge_sort_products()	Call merge_sort(self.products) from algorithms.py. The products list is sorted in place by product ID.	None
quick_sort_products()	Call quick_sort(self.products) from algorithms.py. The products list is sorted in place by product ID.	None
quick_select_product(k: int)	Call quick_select(self.products, k) from algorithms.py and return the Product it returns.	Product | None

Method	Purpose	Return type
	Return None for an invalid rank. QuickSelect may rearrange the product list.	

Important: do not reimplement Merge Sort, Quick Sort, QuickSelect, or hashing inside Store. The Store methods are the application-level interface; the algorithms and mapping logic belong in algorithms.py and the HashMapping class.

5. Example Input, Output, and Expected Behavior

In this project, “input” means method calls and objects passed to your functions or classes. “Output” means returned values or observable changes in the objects. You do not need to use input().

Example 1: Divide-and-Conquer Sorting and QuickSelect

```
store = Store()
mouse = Product("P300", "Mouse", 29.99)
keyboard = Product("P100", "Keyboard", 59.99)
headphones = Product("P200", "Headphones", 39.99)

store.add_product(mouse)
store.add_product(keyboard)
store.add_product(headphones)

store.merge_sort_products()
print([p.get_id() for p in store.products])  # Output: ['P100', 'P200', 'P300']

print(store.quick_select_product(1).get_id())  # Output: P200
```

Example 2: HashMapping and Product Lookup

```
mapping = HashMapping(size=3)
mapping.put("P100", "Keyboard")
mapping.put("P200", "Headphones")

print(mapping.get("P100"))     # Output: Keyboard
print("P200" in mapping)        # Output: True
print(len(mapping))             # Output: 2

mapping.put("P100", "Wireless Keyboard")
print(mapping.get("P100"))     # Output: Wireless Keyboard

try:
    mapping.get("P999")
except KeyError:
    print("KeyError")             # Output: KeyError
```

Example 3: Collision Handling and Rehashing

A collision occurs when two different keys produce the same bucket index. With a small table, collisions are easy to create and inspect. The expected behavior is that both key-value pairs remain available.

```
mapping = HashMapping(size=3)
mapping.put("A", 10)
mapping.put("D", 20)
```

```
print(mapping.get("A"))          # Output: 10
print(mapping.get("D"))          # Output: 20
print(len(mapping))               # Output: 2

# Adding more entries may trigger rehashing.
mapping.put("G", 30)
mapping.put("J", 40)

print(mapping.get("A"))          # Output: 10
print(mapping.get("J"))          # Output: 40
```

The exact bucket used by a string key depends on Python's hash() value, so do not assume a particular bucket number in your tests. Test the observable behavior: all inserted keys can still be retrieved after collisions and after rehashing.

Example 4: Store Product Lookup Uses HashMapping

```
store = Store()
mouse = Product("P300", "Mouse", 29.99)
keyboard = Product("P100", "Keyboard", 59.99)

store.add_product(mouse)
store.add_product(keyboard)

print(store.find_product("P100").get_name())  # Output: Keyboard
print(store.find_product("P999"))               # Output: None
```

The Store keeps its product list for the catalog, but find_product() uses the HashMapping so lookup does not require scanning the entire list.

6. Required Testing with unittest

Submit test_milestone4.py using Python's built-in unittest framework. Test observable behavior, not private implementation details.

- Import unittest and the functions/classes you are testing.
- Create test classes that inherit from unittest.TestCase.
- Every test method must start with test_.
- Use assertions to check returned values and observable behavior.
- Do not inspect private variables inside algorithms, HashMapping, or Store implementation.
- Test the algorithms directly and test Store methods as integration behavior.
- Include normal and edge cases: empty lists, one product, duplicates, already-sorted input, reverse-sorted input, valid/invalid QuickSelect ranks, missing keys, updates, collisions, and rehashing.

Test area	Minimum tests	Examples
Merge Sort	3	empty/one item, reverse order, duplicates
Quick Sort	3	empty/one item, reverse order, duplicates
QuickSelect	3	first rank, middle rank, invalid rank
HashMapping	3	put/get, update, collision or rehash behavior

Test area	Minimum tests	Examples
Store integration	3	sorting, QuickSelect, hash-based find_product

Unit-test expectation
Target: at least 15 meaningful tests. The unit-test file is worth 10 points and will be reviewed manually by the TA. Meaningful coverage and correct use of unittest matter more than simply reaching the minimum count.

7. Running-Time Analysis

Submit a plain-text file named complexity.txt. For each operation below, provide its requested worst-case or expected running time. Use the exact operation names shown here.

- merge_sort (worst case)
- quick_sort (average case)
- quick_sort (worst case)
- quick_select (average case)
- quick_select (worst case)
- HashMapping.put (average case)
- HashMapping.get (average case)
- HashMapping.get (worst case)
- HashMapping.rehash
- Store.find_product (worst case)

For the hash-based operations, remember the distinction between expected/average behavior with a well-distributed hash function and the worst case when many keys collide. For QuickSort and QuickSelect, distinguish average/best behavior from the worst case caused by consistently poor pivots.

8. Gradescope Submission

This milestone is cumulative. Submit the working Milestone 3 files plus the new algorithms, mapping, tests, and complexity files.

File	Status in M4	What it contains
product.py	Same as M3	Product class from previous milestones.
customer.py	Same as M3	Customer class from previous milestones.
cart.py	Same as M3	ShoppingCart behavior from previous milestones.
order.py	Same as M3	Order class from Milestone 2.
node.py	Same as M3	Node used by LinkedList.
linked_list.py	Same as M3	LinkedList implementation.
stack.py	Same as M3	Stack ADT using LinkedList.
order_queue.py	Same as M3	Queue ADT using LinkedList.
store.py	Updated	Maintains HashMapping and adds the three new Store methods.

File	Status in M4	What it contains
algorithms.py	Updated	Contains Merge Sort, Quick Sort, QuickSelect, and previous required algorithms.
entry.py	New	Entry class for one key-value pair.
hash_mapping.py	New	HashMapping with buckets, separate chaining, and rehashing.
test_milestone4.py	New	Your unittest tests; manually graded.
complexity.txt	New	Ten required running-time answers; manually graded.

Gradescope submissions: You may submit as many times as you would like before the deadline. Gradescope will automatically grade program functionality and provide feedback after each submission. The unittest file and complexity.txt are reviewed manually by the TA. Your final submission before the deadline is used for grading.

9. Code Documentation

Include a short, meaningful docstring for each class and required function. Use brief comments when needed to explain important or non-obvious code. Keep comments focused; do not simply restate every line of code.

10. Post-Milestone Feedback & Peer Evaluation

After each milestone, every student must complete an individual contribution and peer-evaluation form describing their own contributions, their teammate's contributions, teamwork and communication, and any resolved or unresolved issues. Although the project is completed as a team, grades may differ based on individual contribution.

11. Grading

Component	Description	Points
Merge Sort	Produces the correct sorted catalog using Merge Sort. Automatically graded.	12
Quick Sort	Produces the correct sorted catalog using Quick Sort. Automatically graded.	12
QuickSelect	Returns the correct product for valid ranks and handles invalid ranks. Automatically graded.	10
HashMapping	Correctly stores, updates, retrieves, and reports key-value pairs, including collision/rehash behavior. Automatically graded.	16
Store integration	Correctly connects sorting, QuickSelect, and HashMapping product lookup to Store. Automatically graded.	10
Algorithm & data-structure implementation / design	Uses the required divide-and-conquer algorithms and hashing design: divide/conquer/combine, partitioning, one-sided QuickSelect recursion, hash buckets, separate chaining, and rehashing. Store methods call	25

Component	Description	Points
	the required algorithms/mapping rather than duplicating their logic. Manually graded.	
Unit tests	Provides at least 15 meaningful unittest tests covering the required areas and testing observable behavior. Manually graded.	10
Running-time analysis	Provides the 10 required running-time answers in complexity.txt. Manually graded.	5

Final Submission Checklist

- All Milestone 3 functionality still works.
- algorithms.py contains Merge Sort, Quick Sort, QuickSelect, and all previous required algorithms.
- QuickSelect returns the requested zero-based rank without fully sorting the catalog.
- HashMapping handles updates, collisions, and rehashing without losing entries.
- Store.find_product() uses HashMapping rather than a linear scan of self.products.
- test_milestone4.py contains at least 15 meaningful tests.
- complexity.txt contains all 10 required answers.
- Every class and required function has a short docstring.

Looking Ahead

The store can now organize large product collections with divide-and-conquer algorithms and retrieve products through a hash-based mapping. In the next milestone, the application will grow to use even more structured data and more powerful ways to organize related information.

Milestone 4 goal

Scale the marketplace by understanding divide-and-conquer design, selection, hashing, collisions, and rehashing—and by connecting those ideas to a working object-oriented system.